TRANSFORMATION · DOMAIN 3 - DATA TRANSFORMATION, CLEANSING, QUALITY

Expectations and quarantining bad data

Section 3. Warn, drop, fail, and the fourth thing that is not an operator.

SECTION 3 of 10 WEIGHT 10% TOPIC Data quality

admin@sthithapragna.com

An expectation asks one question of every row. **What you do with a no is the whole design decision.**

Three components, every time

- A **name**, used to track and report the expectation. It must be **unique within a dataset**.
- A **constraint**: a SQL Boolean expression evaluated per record.
- An **action** deciding what happens to a failing record.
- The same expectation can be **reused across datasets** in a pipeline, which is how you keep rules consistent.

> ON THE EXAM
>
> Expectations can be defined on streaming tables, materialized views and temporary views. Nothing else supports them.

What the expression cannot do

- No **custom Python functions**. The constraint is SQL, evaluated by the engine.
- No **external service calls**. You cannot check a record against an outside system.
- No **subqueries referencing other tables**. Validation is per-record and self-contained.
- Everything else is fair game: SQL functions, CASE expressions, complex Boolean logic.

> ON THE EXAM
>
> Validation against another table is a join, not an expectation. If a question needs a lookup, expectations are not the mechanism.

Three responses to a failing record

ACTION	WHAT HAPPENS
warn, the default	The invalid record is written to the target, and counted in metrics.
drop	The record is discarded before writing. The count is logged.
fail	The update fails. The transaction is atomically rolled back.

ON THE EXAM

Warn being the default surprises people. An expectation with no violation clause does not stop bad data; it lets it through and counts it.

The decision in one question

- Should the invalid row **stay in the target**? Use warn. You want visibility, not enforcement.
- Should the row **disappear** while processing continues? Use drop.
- Should invalid data **block the update**? Use fail. Any bad record means a serious upstream problem.
- Should the row be **kept somewhere else for inspection**? That is quarantine, and none of the three does it.

> ON THE EXAM
>
> The fourth row is the one the objective names. It is built from the other pieces rather than selected from a list.

Quarantine is a pattern, not an operator

- There is no expect-or-quarantine action. The API has exactly **three**: warn, drop, fail.
- The docs describe quarantining as **advanced logic you implement**, routing invalid records to a separate table.
- A Databricks **Labs** metadata framework does define such a rule type. That is not the core product.
- So expect the name to appear as a plausible distractor, and know it is not a built-in operator.

> ON THE EXAM
>
> The objective says 'develop a quarantining process'. That wording is accurate: you develop it, you do not switch it on.

How a quarantine is actually assembled

- Define the rule **once** as a condition, then use it in both directions.
- The clean table keeps records **where the condition holds**. The quarantine table keeps those where it does not.
- Add the reason for rejection so somebody can triage without re-deriving it.
- Nothing is lost, nothing blocks, and bad records are visible in one place.

> ON THE EXAM
>
> Two datasets reading the same source with opposite filters. That is the whole trick, and it is why it counts as a pattern rather than a feature.

Where the numbers come from, and where they do not

- Metrics are recorded for **warn and drop**, and shown on the Data quality tab of the pipeline.
- **Not for fail.** Because the update fails, nothing is aggregated to report.
- Metrics are also queryable from the **pipeline event log**, which is how you track trends over time.
- For standalone datasets created in Databricks SQL, the Data quality tab is **not available**. Use the event log.

> ON THE EXAM
>
> Fail gives you a detailed error naming the offending record, and no aggregate metrics. Those are different kinds of information.

Fail behaves differently by pipeline mode

- In a **triggered** pipeline, one flow failing does **not** fail other parallel flows.
- In a **continuous** pipeline, the failure stops that flow **and all dependent flows**.
- For real orchestration control, split validation and downstream work into **separate pipelines**, coordinated by task control flow.
- After a failure, the **pipeline code must be fixed** before re-running. It will not simply retry past it.

> ON THE EXAM
>
> Same action, different blast radius depending on the pipeline mode. That asymmetry is worth remembering.

Grouping is Python only

- Both languages support several expectations on one dataset.
- Only **Python** lets you group them and apply a **collective action**.
- The grouped forms take a **dictionary** of name to constraint, so one dictionary serves many datasets.
- That is how the same rule set gets applied with warn at bronze and fail at gold.

> ON THE EXAM
>
> One dictionary, three severities. Warn where you are observing, drop where you are cleaning, fail where you are publishing.

The other half of the objective

- Outside pipelines, the equivalent is Auto Loader's **rescued data column**, from module 11.
- It captures a **missing column, a type mismatch or a case mismatch**, with the source file path.
- Filtering on that column not being null gives you a quarantine table for structural problems.
- It catches **malformed** records, not **invalid** ones. A negative price parses perfectly well.

> ON THE EXAM
>
> Two different failure classes. Rescued data catches records that did not parse; expectations catch records that parsed and are wrong.

Where expectations do not reach

- Only streaming tables, materialized views and temporary views. Nothing else.
- **Not supported for sinks**, which is the module 7 point: writing outside the pipeline loses the quality mechanism.
- **Not supported with AUTO CDC FROM SNAPSHOT.**
- Views are computed only when queried, so metrics may be missing or may appear several times over.

> ON THE EXAM
>
> The sink limitation matters most in design questions. Quality checks stop at the boundary where you leave the pipeline's own datasets.

What to carry into the exam

- **Warn is the default.** An expectation with no violation clause lets bad data through and counts it.
- Drop discards, fail rolls back the transaction atomically.
- **No metrics for fail**, because the update did not complete.
- **Quarantine is a pattern you build**, not an action you select.
- Grouped actions are **Python only**. Rescued data is the classic-jobs equivalent for malformed records.

> ON THE EXAM
>
> That completes Section 3 at 10%. Section 4, Data Sharing and Federation, is next and is the smallest section on the exam.

> A team adds an expectation to a bronze streaming table checking that an order amount is not negative. They write CONSTRAINT non_negative_amount EXPECT (amount >= 0) with no further clause.
>
> A week later, an analyst finds negative amounts in the bronze table. The team insists the expectation is working, and the metrics do show violations being counted.

Who is right?

A The team is right; the expectation works, and warn is the default action, so invalid records are still written.

B The analyst is right; the expectation is malformed and is being ignored entirely.

C The team is right; the negative amounts must have been written before the expectation was added.

D The analyst is right; expectations do not apply to streaming tables.

◆ PAUSE AND DECIDE

Who is right?

A The team is right; the expectation works, and warn is the default action, so invalid records are still written.

Both observations are correct. Warn counts violations and writes the record anyway.

B The analyst is right; the expectation is malformed and is being ignored entirely.

The metrics prove it is being evaluated.

C The team is right; the negative amounts must have been written before the expectation was added.

Plausible timing argument, and the metrics show ongoing violations.

D The analyst is right; expectations do not apply to streaming tables.

Streaming tables are one of the three types that do support them.

ON THE EXAM

Both parties observed something true. The expectation is working exactly as written, and as written it does not block anything.

> A regulated pipeline must never publish a record with a missing customer identifier. The compliance team must also review every rejected record and understand why it failed.
>
> An engineer proposes ON VIOLATION DROP ROW, since it prevents bad records reaching the target.

What is the problem with that proposal?

A. Nothing. Drop prevents publication and the metrics record how many were dropped.

B. Drop discards the records entirely, so there is nothing for compliance to review. A quarantine pattern is needed.

C. Drop is not supported on streaming tables; only warn and fail are.

D. Drop should be replaced with expect_or_quarantine, which routes rejects to a quarantine table.

◆ PAUSE AND DECIDE

What is the problem with that proposal?

A. Nothing. Drop prevents publication and the metrics record how many were dropped.

Half the requirement. A count is not a reviewable record.

B. Drop discards the records entirely, so there is nothing for compliance to review. A quarantine pattern is needed.

Two requirements: block publication, and preserve rejects for review. Drop satisfies only the first.

C. Drop is not supported on streaming tables; only warn and fail are.

All three actions are supported.

D. Drop should be replaced with expect_or_quarantine, which routes rejects to a quarantine table.

No such operator exists in the core API. It comes from a Databricks Labs framework.

ON THE EXAM

D is the distractor this module exists to inoculate against. The name sounds exactly right and the operator is not real.

> A gold table uses ON VIOLATION FAIL UPDATE on a critical constraint. Overnight the pipeline failed. The on-call engineer opens the Data quality tab expecting to see how many records violated the constraint, and finds no metrics for that run at all.
>
> They conclude the monitoring is broken.

What should they be told?

A Metrics are not recorded for fail, because the update did not complete. The error message names the offending record.

B Metrics take up to 24 hours to appear in the Data quality tab.

C The pipeline must be configured with a metrics reporter before any expectation metrics appear.

D Fail records metrics only in the event log, not the UI. Query the event log for the counts.

◆ PAUSE AND DECIDE

What should they be told?

A Metrics are not recorded for fail, because the update did not complete. The error message names the offending record.

Documented behaviour. Fail gives a detailed violation message instead of aggregate counts.

B Metrics take up to 24 hours to appear in the Data quality tab.

Invented delay.

C The pipeline must be configured with a metrics reporter before any expectation metrics appear.

A real setting, and it would have suppressed warn and drop metrics too, which the team has been seeing.

D Fail records metrics only in the event log, not the UI. Query the event log for the counts.

There are no aggregate counts to find, in either place.

ON THE EXAM

Fail trades a count for a specific record. That is usually the better trade when diagnosing, and it is not what the tab is for.

> A team wants an expectation that rejects any order whose customer identifier does not appear in the customers dimension table. They write a constraint containing a subquery against that table.
>
> The pipeline will not start.

Why, and what should they do instead?

A. Subqueries referencing other tables are not allowed in a constraint. Use a join, or an anti join to isolate orphans.

B. The subquery is allowed but must be wrapped in a Python UDF for the pipeline to evaluate it.

C. The customers table must be declared as a temporary view before a constraint can reference it.

D. Constraints can only reference columns of the same name in both tables.

◆ PAUSE AND DECIDE

Why, and what should they do instead?

A Subqueries referencing other tables are not allowed in a constraint. Use a join, or an anti join to isolate orphans.

A stated limitation. Cross-table validation is a join, and module 12's anti join is the natural way to find orphans.

B The subquery is allowed but must be wrapped in a Python UDF for the pipeline to evaluate it.

Custom Python functions are also explicitly disallowed in a constraint.

C The customers table must be declared as a temporary view before a constraint can reference it.

The restriction is on referencing other tables at all, not on how they are declared.

D Constraints can only reference columns of the same name in both tables.

Invented rule.

ON THE EXAM

Expectations validate a record against itself. Anything requiring another dataset is a join, and belongs in the query rather than the constraint.

> A pipeline defines the same eight validation rules on bronze, silver and gold datasets. On bronze the team wants visibility only, on silver they want bad rows removed, and on gold nothing invalid may pass.
>
> The pipeline is currently written in SQL, and the rules are copied out three times.

What is the better approach?

A Keep SQL and use a different ON VIOLATION clause on each copy of the rules.

B Write the datasets in Python and apply one dictionary of rules with expect_all, expect_all_or_drop and expect_all_or_fail.

C Define the rules once in SQL and reference them from the other datasets by name.

D Use a single gold-level fail expectation, since bronze and silver feed it anyway.

◆ PAUSE AND DECIDE

What is the better approach?

A. Keep SQL and use a different ON VIOLATION clause on each copy of the rules.
Works, and leaves three copies of eight rules to drift apart.

B. Write the datasets in Python and apply one dictionary of rules with expect_all, expect_all_or_drop and expect_all_or_fail.
One definition, three severities. Collective actions are Python only, which is exactly why Python is the answer here.

C. Define the rules once in SQL and reference them from the other datasets by name.
SQL supports multiple expectations but has no mechanism for a shared, named rule set with a collective action.

D. Use a single gold-level fail expectation, since bronze and silver feed it anyway.
Loses the graduated response, and a gold failure would then be the first sign of a bronze problem.

ON THE EXAM

This is the case where language choice is driven by a capability rather than preference. Grouped actions do not exist in SQL.

> A classic job uses Auto Loader to ingest JSON into a bronze table, with a rescued data column enabled. The team wants to catch two things: records whose fields did not parse, and orders with a negative total.
>
> They plan to rely on the rescued data column for both.

What is wrong with that plan?

A Nothing. The rescued data column captures any record that fails validation.

B Rescued data catches records that did not parse. A negative total parses fine, so it needs a separate check.

C Rescued data works only for CSV, not JSON.

D The rescued data column must be paired with an expectation to function at all.

◆ PAUSE AND DECIDE

What is wrong with that plan?

A. Nothing. The rescued data column captures any record that fails validation.
It captures parsing failures, not business-rule failures.

B. Rescued data catches records that did not parse. A negative total parses fine, so it needs a separate check.
Two different failure classes. Structural problems go to rescued data; a valid number that is wrong does not.

C. Rescued data works only for CSV, not JSON.
It works across the inference-based formats, JSON included.

D. The rescued data column must be paired with an expectation to function at all.
It is an Auto Loader feature and works independently.

ON THE EXAM

Malformed and invalid are different problems. A quarantine that catches only one of them will look like it is working.